

HR Database Normalization

Original HR Table (Reference)

The company stores all employee HR data in one flat table:

emp_id	emp_name	department	manager	skills	certifications	project	project_location
E01	Aung Aung	IT	U Hla	Python, SQL	AWS, Azure	HRIS	Yangon
E02	Su Su	Finance	Daw May	Excel, Power BI	CPA	Payroll	Mandalay
E03	Kyaw Kyaw	IT	U Hla	Java, Python	AWS	HRIS	Yangon

Q1. What normalization problem exists in the original HR table?

The original table violates the First Normal Form (1NF) and contains several other normalization problems:

- Repeating Groups / Multi-valued Attributes: The 'skills' column contains multiple values (e.g., 'Python, SQL') and the 'certifications' column also holds multiple values (e.g., 'AWS, Azure'). A single cell should hold only one atomic value.
 - Partial Dependency (2NF violation): Non-key attributes like emp_name, department, and manager depend only on emp_id (part of a composite key), not on the full composite key (emp_id + skill + certification).
 - Transitive Dependency (3NF violation): The manager column depends on department, not directly on emp_id. This means: emp_id -> department -> manager.
 - Multi-valued Dependency (4NF violation): An employee's skills and certifications are independent of each other but are stored together, causing redundant rows.
 - Data Redundancy: Employee details (name, department, manager, project, location) are repeated across multiple rows for each skill/certification combination.
-

Q2. Convert the HR table into 1NF

First Normal Form (1NF) requires:

- Each column contains atomic (single) values
- Each row is unique
- No repeating groups or arrays

Resolution: Split multi-valued 'skills' and 'certifications' into separate rows. The composite primary key becomes (emp_id, skill, certification).

emp_id	emp_name	department	manager	skill	certification	project	project_location
E01	Aung Aung	IT	U Hla	Python	AWS	HRIS	Yangon
E01	Aung Aung	IT	U Hla	Python	Azure	HRIS	Yangon
E01	Aung Aung	IT	U Hla	SQL	AWS	HRIS	Yangon
E01	Aung Aung	IT	U Hla	SQL	Azure	HRIS	Yangon
E02	Su Su	Finance	Daw May	Excel	CPA	Payroll	Mandalay
E02	Su Su	Finance	Daw May	Power BI	CPA	Payroll	Mandalay
E03	Kyaw Kyaw	IT	U Hla	Java	AWS	HRIS	Yangon
E03	Kyaw Kyaw	IT	U Hla	Python	AWS	HRIS	Yangon

Note: The composite primary key is (emp_id, skill, certification) to ensure uniqueness of each row.

Q3. What is the main issue after converting to 1NF?

After converting to 1NF, the table suffers from Partial Dependencies, which violates 2NF. Specifically:

- emp_name depends only on emp_id (not on the full composite key emp_id + skill + certification)
- department depends only on emp_id
- manager depends only on emp_id
- project depends only on emp_id
- project_location depends only on emp_id

This causes significant data redundancy. For example, Aung Aung's department and manager are repeated in every row for every skill and certification combination. Updating one record risks creating inconsistencies (update anomaly).

Q4. Convert the 1NF table into 2NF

Second Normal Form (2NF) requires: The table is in 1NF AND every non-key attribute is fully dependent on the entire primary key (no partial dependencies).

Resolution: Decompose the table by moving partially dependent attributes into separate tables.

Table 1: Employee (Primary Key: emp_id)

emp_id (PK)	emp_name	department	manager	project	project_location	
E01	Aung Aung	IT	U Hla	HRIS	Yangon	
E02	Su Su	Finance	Daw May	Payroll	Mandalay	
E03	Kyaw Kyaw	IT	U Hla	HRIS	Yangon	

Table 2: Employee_Skill (Primary Key: emp_id + skill)

emp_id (FK)	skill	
E01	Python	

E01	SQL	
E02	Excel	
E02	Power BI	
E03	Java	
E03	Python	

Table 3: Employee_Certification (Primary Key: emp_id + certification)

emp_id (FK)	certification	
E01	AWS	
E01	Azure	
E02	CPA	
E03	AWS	

Now, all non-key attributes in each table are fully dependent on that table's complete primary key.

Q5. What issue still exists after 2NF?

After achieving 2NF, the Employee table still contains a Transitive Dependency, which violates 3NF:

- emp_id -> department (direct dependency)
- department -> manager (manager depends on department, not on emp_id)
- Therefore: emp_id -> department -> manager (transitive dependency)

Additionally, project_location depends on project (not on emp_id directly):

- emp_id -> project (direct)
- project -> project_location (transitive)

This means if the same department appears for multiple employees, the manager name is stored repeatedly. If the manager changes, all rows must be updated — a classic update anomaly.

Q6. Convert the 2NF design into 3NF

Third Normal Form (3NF) requires: The table is in 2NF AND no non-key attribute is transitively dependent on the primary key.

Resolution: Remove transitive dependencies by extracting department and project data into their own tables.

Table 1: Employee (Primary Key: emp_id)

emp_id (PK)	emp_name	dept_id (FK)	project_id (FK)		
E01	Aung Aung	D01	P01		
E02	Su Su	D02	P02		
E03	Kyaw Kyaw	D01	P01		

Table 2: Department (Primary Key: dept_id)

dept_id (PK)	department	manager	
D01	IT	U Hla	
D02	Finance	Daw May	

Table 3: Project (Primary Key: project_id)

project_id (PK)	project_name	location	
P01	HRIS	Yangon	
P02	Payroll	Mandalay	

Employee_Skill and Employee_Certification tables remain unchanged from 2NF.

Now every non-key attribute in each table depends directly and only on that table's primary key.

Q7. Is the HR database now in BCNF?

Boyce-Codd Normal Form (BCNF) is a stricter version of 3NF. It requires: For every functional dependency $X \rightarrow Y$, X must be a superkey.

Assessment: Yes, the HR database is effectively in BCNF after the 3NF conversion, because:

- Employee table: emp_id (superkey) $\rightarrow$ emp_name, dept_id, project_id. All determinants are superkeys.
- Department table: dept_id (superkey) $\rightarrow$ department, manager. All determinants are superkeys.
- Project table: project_id (superkey) $\rightarrow$ project_name, location. All determinants are superkeys.
- Employee_Skill: (emp_id, skill) composite key is the only determinant. Satisfies BCNF.
- Employee_Certification: (emp_id, certification) composite key is the only determinant. Satisfies BCNF.

Potential BCNF issue to watch: If a manager can only manage one department, then manager $\rightarrow$ department would be an additional functional dependency. If we also have emp_id $\rightarrow$ manager, then manager is a determinant but not a superkey. In that scenario, the Department table would need further decomposition. However, with the current schema design, BCNF is satisfied.

Q8. What 4NF issue can exist in HR data?

Fourth Normal Form (4NF) requires: The table is in BCNF AND there are no non-trivial multi-valued dependencies.

The 4NF issue in HR data arises from Multi-valued Dependencies (MVDs). Specifically:

An employee can have multiple skills AND multiple certifications that are completely independent of each other:

- emp_id $\twoheadrightarrow$ skill (an employee multi-determines skills independently)
- emp_id $\twoheadrightarrow$ certification (an employee multi-determines certifications independently)

If skills and certifications were stored in the same table, every combination of skill and certification would need a row, causing unnecessary redundancy. For example:

- E01 | Python | AWS
- E01 | Python | Azure
- E01 | SQL | AWS
- E01 | SQL | Azure

Resolution (already applied in 2NF): Store Employee_Skill and Employee_Certification as separate tables. This eliminates the multi-valued dependency and brings the design to 4NF.

Q9. What is a possible 5NF issue in HR data?

Fifth Normal Form (5NF), also called Project-Join Normal Form (PJNF), requires: The table cannot be decomposed further without losing information (no join dependencies that are not implied by candidate keys).

A possible 5NF issue in HR data involves a three-way relationship between employees, skills, and projects. Consider this business rule:

'An employee can use a particular skill on a particular project, but not every skill is used on every project they work on.'

This creates a three-way join dependency:

- Employee E01 uses Python on project HRIS
- Employee E01 uses SQL on project HRIS
- Employee E01 uses Python on project X (hypothetically), but NOT SQL on project X

In such a case, a three-way table (emp_id, skill, project_id) cannot be losslessly decomposed into any combination of two-way tables without losing information about which employee uses which skill on which project.

Resolution: Create an Employee_Project_Skill junction table with the composite key (emp_id, skill, project_id) to represent this three-way relationship directly, ensuring no 5NF violation.

Q10. What is the final normalized HR database design?

The final fully normalized HR database (satisfying 1NF through 5NF) consists of the following tables:

Table	Primary Key	Columns
Employee	emp_id	emp_name, dept_id (FK), project_id (FK)
Department	dept_id	department_name, manager
Project	project_id	project_name, location

Employee_Skill	emp_id + skill	emp_id (FK), skill
Employee_Certification	emp_id + certification	emp_id (FK), certification
Employee_Project (optional)	emp_id + project_id	emp_id (FK), project_id (FK), role

Summary of Each Table

1. Employee Table (emp_id PK)

- Stores core employee information
- Links to Department via dept_id (FK) and Project via project_id (FK)
- Eliminates transitive dependencies from 3NF conversion

2. Department Table (dept_id PK)

- Resolves transitive dependency: department -> manager
- Each department has exactly one manager stored in one place

3. Project Table (project_id PK)

- Resolves transitive dependency: project -> project_location
- Each project's location is stored once

4. Employee_Skill Table (emp_id + skill composite PK)

- Resolves partial dependency from 2NF conversion
- Resolves multi-valued dependency (4NF): skills are independent of certifications
- Allows an employee to have many skills

5. Employee_Certification Table (emp_id + certification composite PK)

- Resolves partial dependency from 2NF conversion
- Resolves multi-valued dependency (4NF): certifications are independent of skills
- Allows an employee to hold many certifications

6. Employee_Project (optional, for 5NF) (emp_id + project_id composite PK)

- Addresses 5NF join dependency between employees, skills, and projects
- Can include a 'role' column to describe what role the employee plays in the project

Normalization Journey Summary

Normal Form	Requirement	Action Taken
1NF	Atomic values, no repeating groups	Split multi-valued skills and certifications into separate rows
2NF	No partial dependencies	Moved emp_name, dept, manager, project to Employee table; created Employee_Skill and Employee_Certification tables
3NF	No transitive dependencies	Extracted Department table (dept->manager) and Project table (project->location)
BCNF	Every determinant is a superkey	Already satisfied after 3NF with current schema
4NF	No multi-valued dependencies	Separate Employee_Skill and Employee_Certification tables (done in 2NF)
5NF	No join dependencies	Optional Employee_Project_Skill table for three-way relationship if needed